

Competitive Programming

Dynamic Programming Intro

Overview

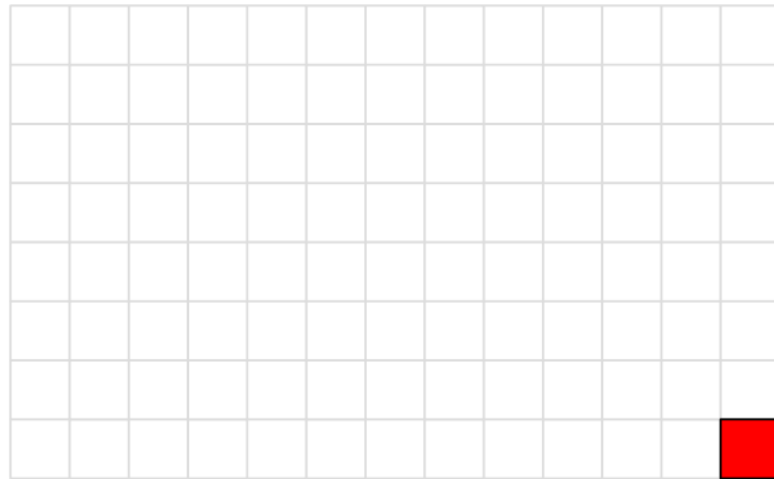
- Lots of algorithms work by:
 - Reducing a larger problem instance
 - ... to smaller instances of the same problem.
- For example, mergesort
 - To sort a big list of elements
 - ... you can sort two smaller lists
- Example, selection
 - To select from a big list
 - ... you can partition and select from a smaller list

Overview

- For algorithms like this, it's common to create a recursive solution
 - Start at the large problem instance
 - Work your way back to smaller instances
 - ... and eventually to base cases that are trivial.
- But, that's not the only way a solution can work
 - Dynamic programming goes the other way,
 - Working from trivial cases
 - ... to larger problem instances
 - ... and eventually to the problem instance you want to solve. Fibonacci Numbers

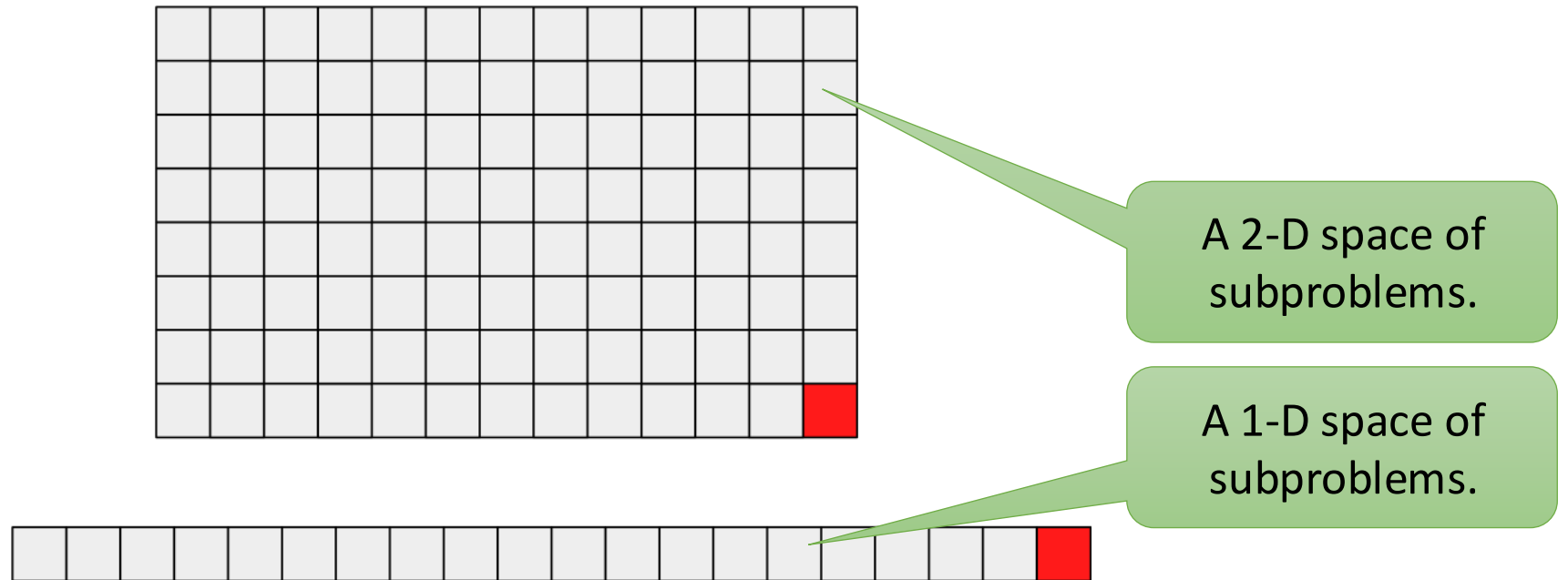
Thinking about Dynamic Programming

- You have a problem instance you want to solve.
 - E.g., how many ways can I move from some start to far-away end location?
 - E.g., what's the best team I can among a large set of available players?



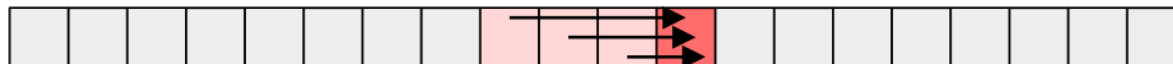
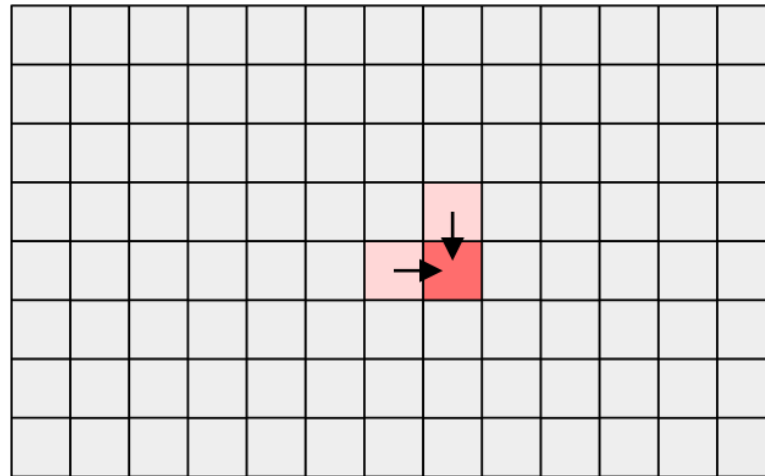
Thinking about Dynamic Programming

- Consider a space of easier versions of that problem instance.
 - E.g., how many ways can I move from some start to nearer end location?
 - E.g., what's the best team I can among a subset of available players?



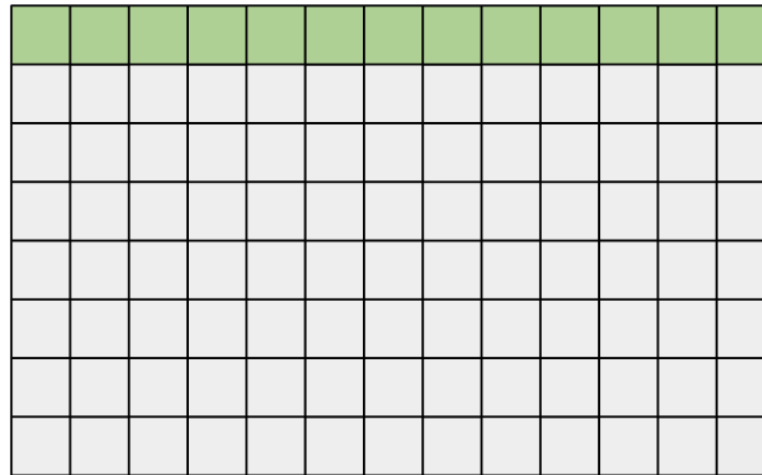
Thinking about Dynamic Programming

- Such that solving smaller problem instances can help make it easy to solve harder problem instances.
 - E.g., get to an intermediate location, then take another step.
 - E.g., make a team with some available players, then consider adding one more.



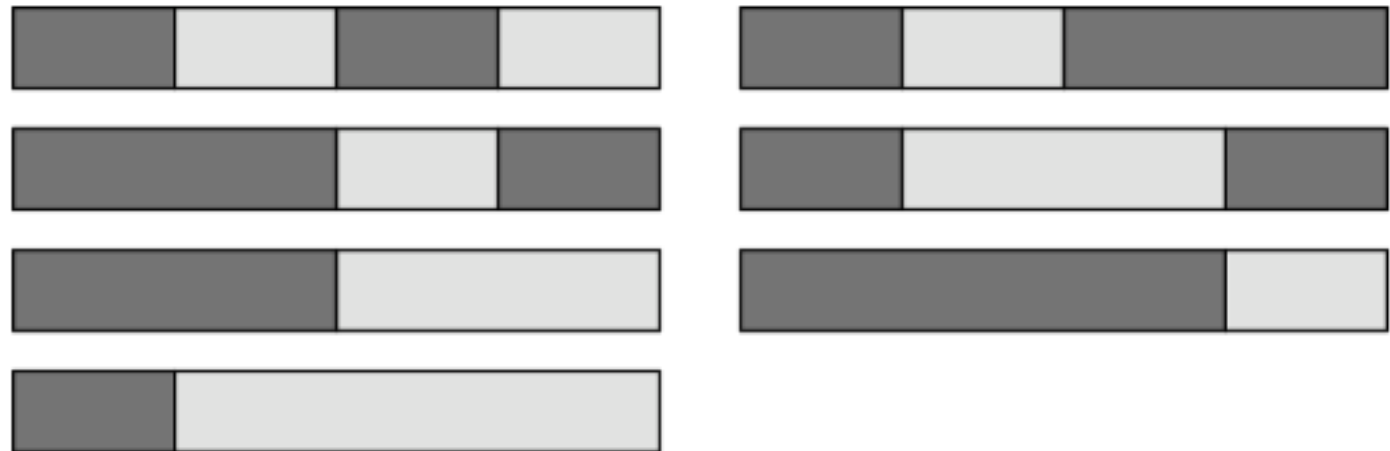
Thinking about Dynamic Programming

- There are some base-case problem instances that are easy to solve.
 - E.g., how many ways to go nowhere.
 - E.g., the best team you can make if no players are available.



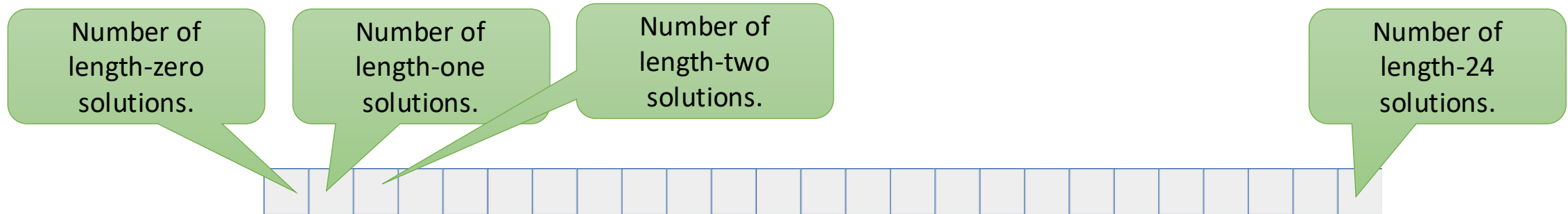
A Competitive Programming Example

- Building a longer plank out of 1-meter, 2-meter and 3-meter sections.
- Report the number of different ways to build the plank.



Space of Sub-Problems

- Solve for all plank lengths



- A one-dimensional space of sub-problems.

Space of Sub-Problems

- Just one length-zero solution (no planks).



- This is the base case.

Space of Sub-Problems

- Fill in element 1.
- Try adding a length one plank.



- Any other plank would be too long.

Space of Sub-Problems

- Fill in element 2.
- Try adding a length one plank to element 1.



- Or adding a length-two plank to element zero.



Space of Sub-Problems

- Fill in element 3.
- Try adding a length one plank to element 2.



- Or a length-two plank to element 1.



- Or, a length-three plank to element zero.



Space of Sub-Problems

- Fill in element 4.

Three horizontal bars representing the distribution of the number of children per family. The first bar has a single dark grey block above the first cell (1 child). The second bar has a single dark grey block above the second cell (2 children). The third bar has a single dark grey block above the third cell (3 children).

Space of Sub-Problems

- Fill in element 5.



1	1	2	4	7	7																							
---	---	---	---	---	---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--



1	1	2	4	7	11																						
---	---	---	---	---	----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

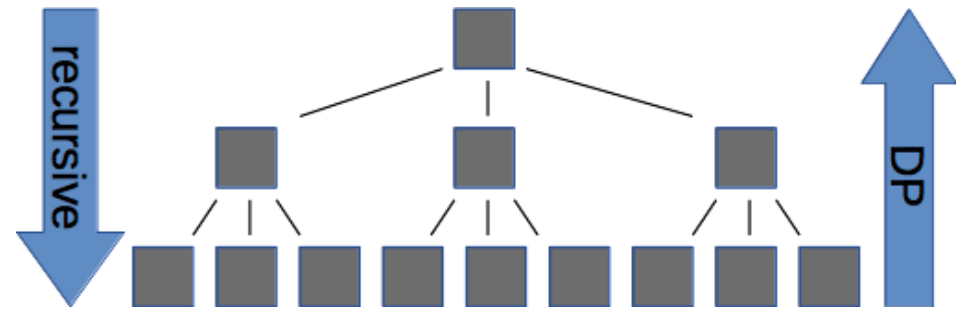


1	1	2	4	7	13																						
---	---	---	---	---	----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

- All the way to element 24.

Recursion vs Dynamic Programming

- Recursive and Dynamic Programming can each have their advantages.
- With dynamic programming, it's easy to re-use subproblem solutions
- With recursion, you may not actually need the solution to some subproblems
- Memoizing tries to capture the advantages of both,
 - but, the cache has some complexity and overhead



Bottom-up Implementation

- A typical DP solution works bottom-up.

```
int T[ n + 1 ];  
T[ 0 ] = 1;
```

Base case, there's one way to build a plank of length 0.

```
for ( int i = 1; i <= n; i++ ) {  
    T[ i ] = T[ i - 1 ];  
    if ( i >= 2 ) T[ i ] += T[ i - 2 ];  
    if ( i >= 3 ) T[ i ] += T[ i - 3 ];  
}
```

Add a length-1 plank to a previous solution.

Add a length-2 plank to a previous solution.

Add a length-3 plank to a previous solution.

```
println( T[ n ] );
```

Add a length-3 plank to a previous solution.

A Top-down Implementation

- A typical recursive solution works top-down

```
int plank( int len ) {  
    if ( len == 0 ) return 1;  
  
    int ans = plank( len - 1 );  
    if ( len >= 2 ) ans += plank( len - 2 );  
    if ( len >= 3 ) ans += plank( len - 3 );  
    return ans;  
}
```

Base case, answer right away.

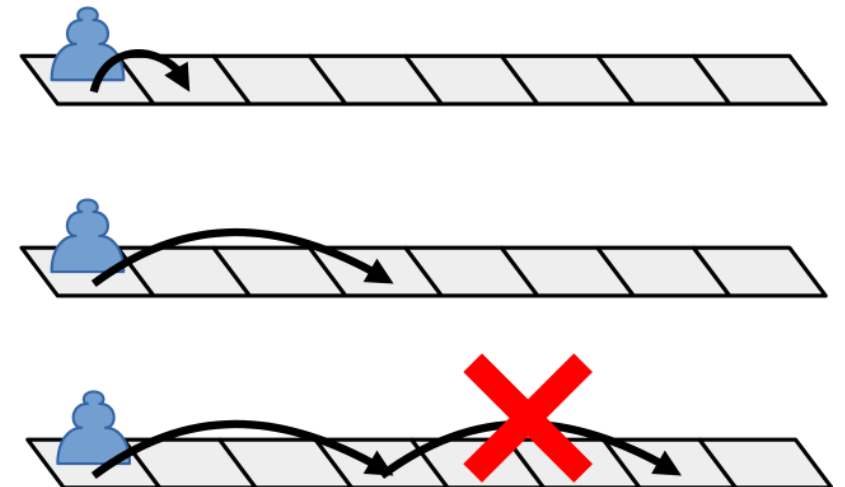
Otherwise, solve up to 3 smaller problems.

This will work ... but not well.
Running time is exponential.

- Caching solutions can make this as fast as DP.
 - That's called memorization.

Jumping Game

- You have a game piece that can move down a line of spaces.
 - It can take a single-hop.
 - It can take a triple-hop, but then it has to rest.
Can't do two triple-hops in a row.
- How many ways are there to move n spaces down the board?

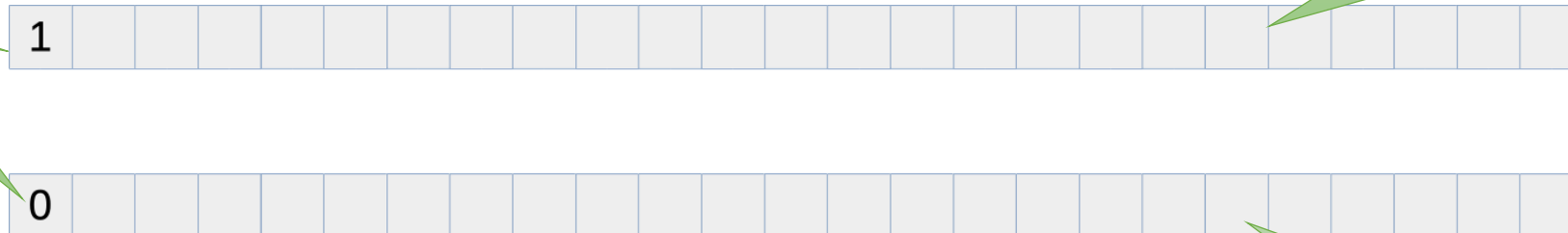


Jumping Game

- We need another dimension in our space of subproblems.
 - To remember whether we just took a triple-hop.
 - This is a 2D DP problem (or really, seems like 1.5D to me).

Base case, how many length-zero paths are there.

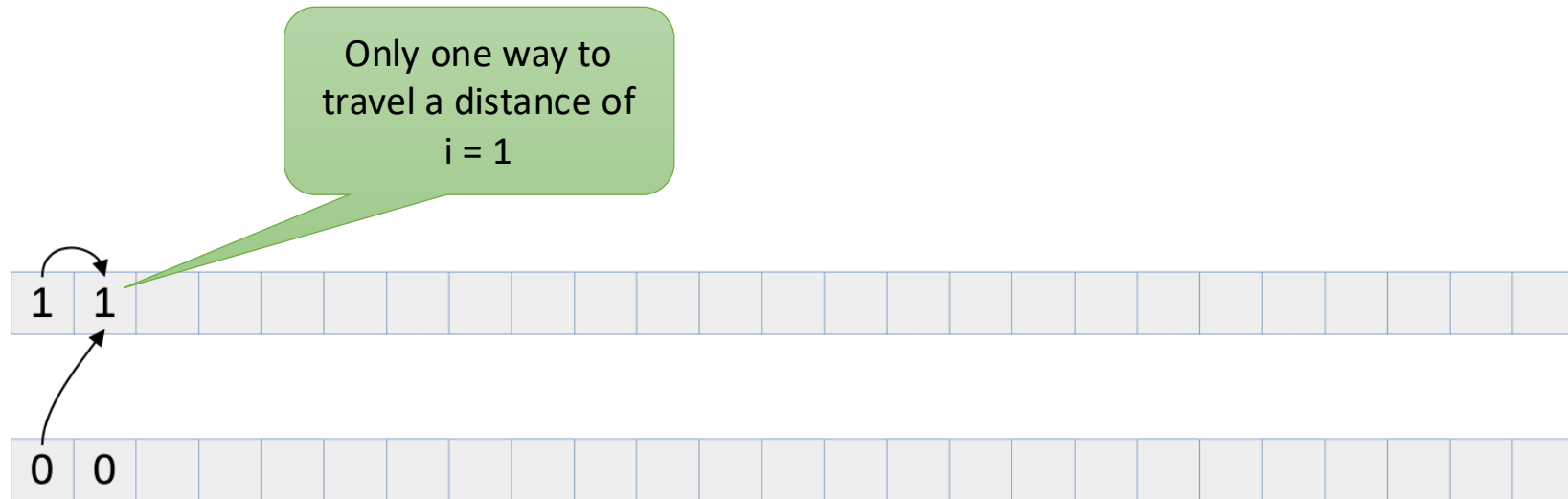
Number of paths not ending in a triple-hop.



Number of paths ending in a three-hop.

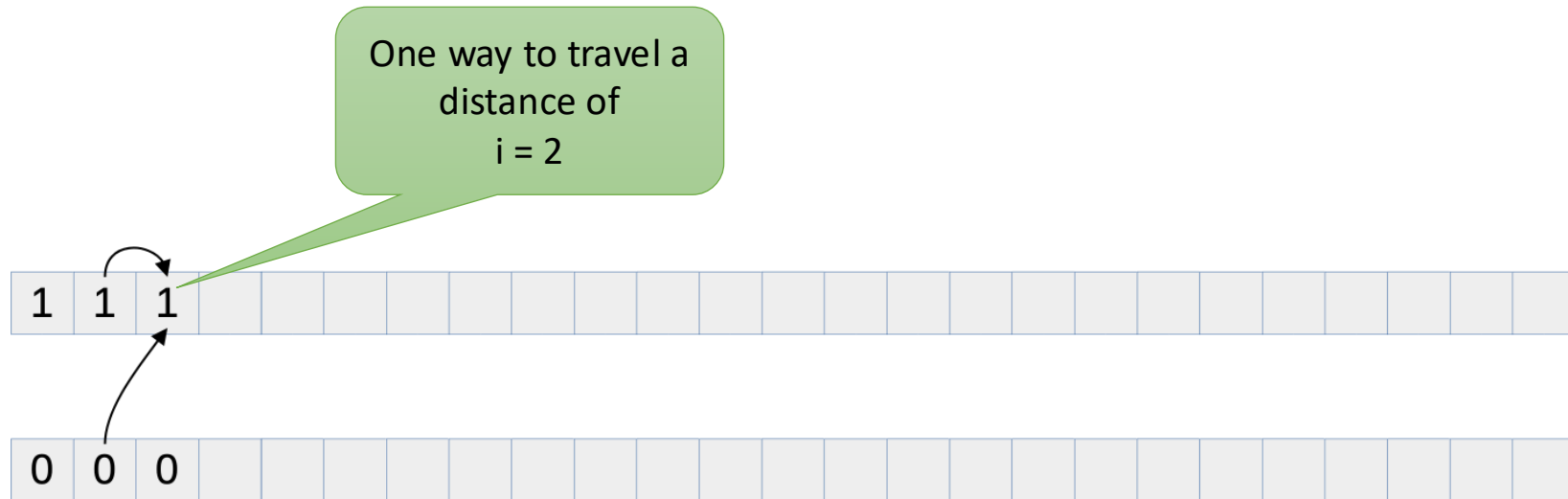
Jumping Game

- Filling in cells left-to-right.



Jumping Game

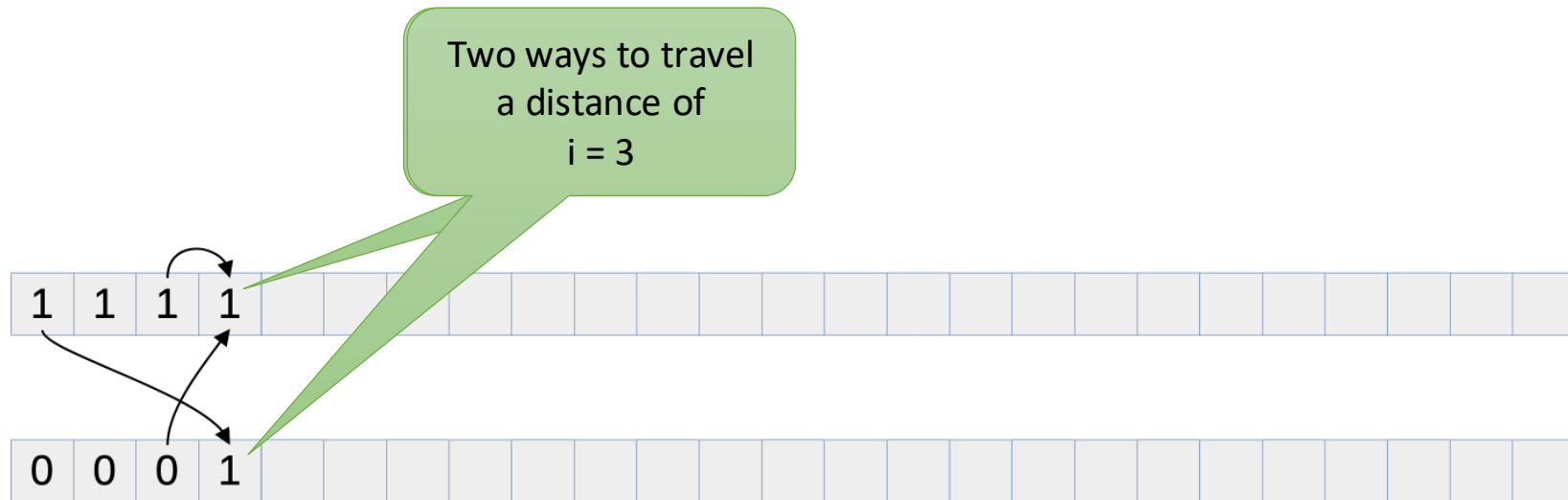
- Filling in cells left-to-right.



One way to travel a distance of $i = 2$

Jumping Game

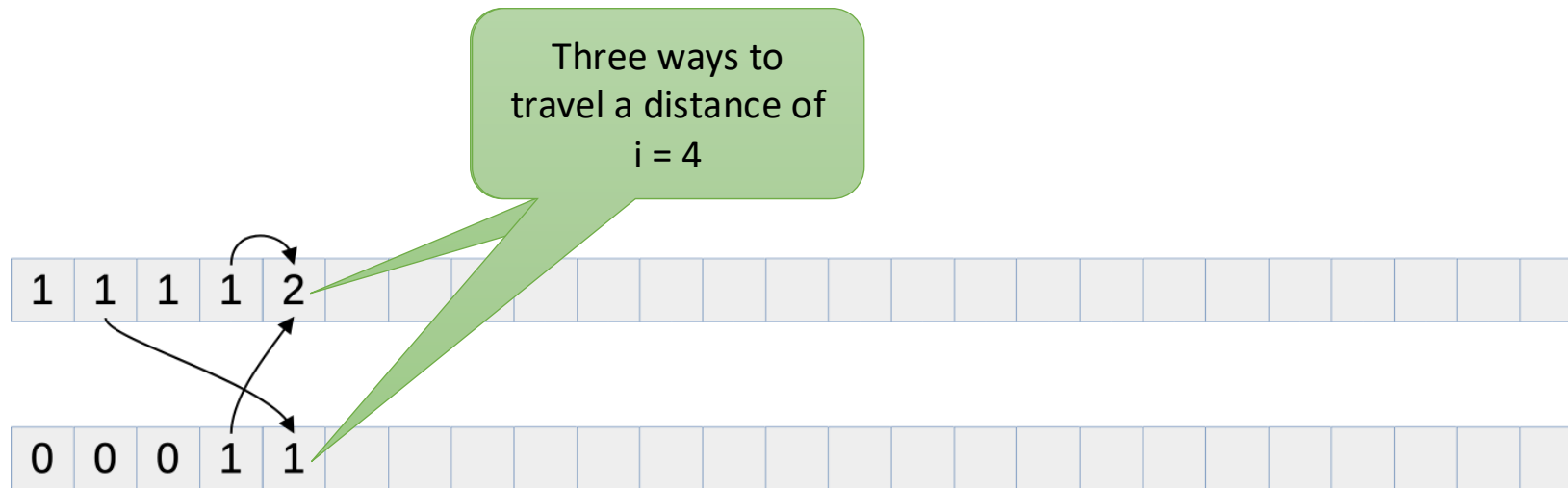
- Filling in cells left-to-right.



Two ways to travel
a distance of
 $i = 3$

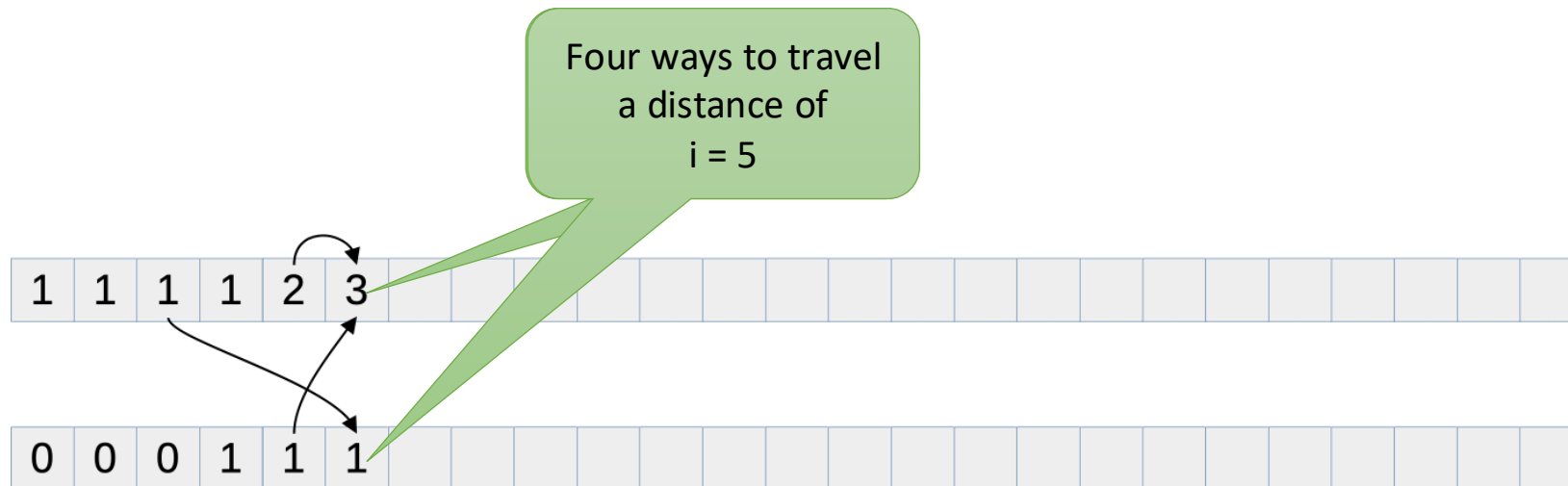
Jumping Game

- Filling in cells left-to-right.



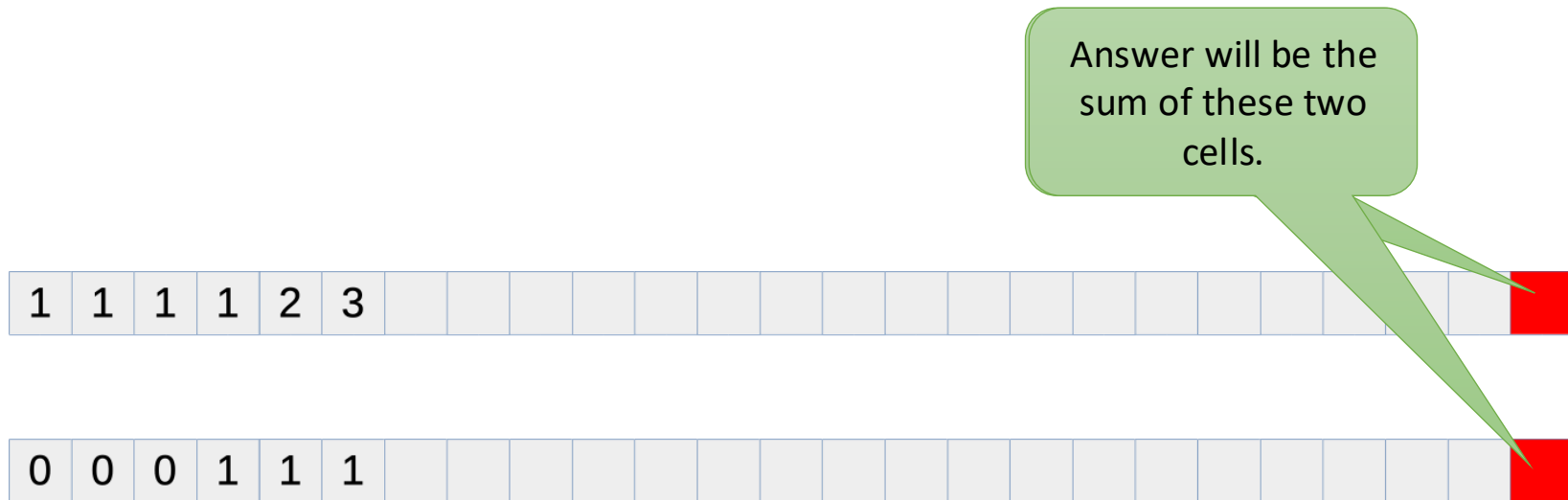
Jumping Game

- Filling in cells left-to-right.



Jumping Game

- Filling in cells left-to-right.



Answer will be the sum of these two cells.